

Bakalářské zkoušky (příklady otázek)

2026-06-22

1 Rozděl a panuj (společné okruhy)

Uvažme následující úlohu: Je dána matice A složená z $n \times n$ celých čísel. Každý její řádek i sloupec tvoří neklesající posloupnost. Chceme zjistit, zda se v této matici nachází 0. Matice je už načtena v paměti, čtení vstupu není potřeba řešit. Načtená vstupní data se nepočítají do prostorové složitosti algoritmu, který úlohu řeší.

Použijeme metodu Rozděl a panuj. Podíváme se na prostřední prvek matice. Pokud je nulový, uspěli jsme. Pokud je kladný, všechny prvky v pravé dolní čtvrtině matice jsou také kladné, takže stačí úlohu řešit rekurzivně ve zbývajících třech čtvrtinách. A pokud je prostřední prvek záporný, můžeme naopak vyloučit levou horní čtvrtinu.

Vaše úkoly:

1. Zapište algoritmus založený na uvedené myšlence. K zápisu můžete použít například jazyk Python, nebo pseudokód podobné úrovně detailu.
2. Analyzujte jeho časovou a prostorovou složitost v závislosti na n . Pokud k tomu používáte nějakou větu, uveďte její znění.
3. Navrhněte efektivnější algoritmus a analyzujte jeho časovou a prostorovou složitost.

Nástin řešení

1. Algoritmus může mít podobu rekurzivní funkce, která dostane levý horní roh a rozměry podmatice, v níž má hledat. V Pythonu ji můžeme zapsat takto:

```
def find(i, j, n, m):
    if n < 1 or m < 1:
        return None
    nn = n // 2
    mm = m // 2
    ci = i + nn
    cj = j + mm
    if A[ci][cj] == 0:
        return True
    elif A[ci][cj] < 0:
        return (find(i+nn, j, n-nn, mm) or
                find(i, j+mm, n, m-mm) or
                find(i+nn, j+mm, n-nn, m-mm))
    else:
        return (find(i, j, nn, mm) or
                find(i+nn, j, n-nn, mm) or
                find(i, j+mm, nn, m-mm))
```

2. Časová složitost je popsána rekurencí $T(n) = 3T(n/2) + \Theta(1)$ s okrajovou podmínkou $T(1) = \Theta(1)$. Jejím řešením podle Kuchařkové věty (viz Průvodce labyrintem algoritmů, kapitola 10.4) je $T(n) \in \Theta(n^{\log_2 3}) \subseteq \mathcal{O}(n^{1.59})$. Prostorová složitost je popsána rekurencí $S(n) = S(n/2) + \Theta(1)$, $S(1) = \Theta(1)$ s řešením $S(n) \in \Theta(\log n)$.
3. Jedna možnost je v každém řádku zvlášť binárně vyhledávat, čímž dosáhneme složitosti $\Theta(n \log n)$.

Nebo se můžeme podívat na levý dolní prvek matice. Pokud je kladný, jsou kladné všechny prvky v posledním řádku, takže můžeme celý poslední řádek smazat. Pokud je naopak záporný, můžeme smazat celý první sloupec. Těchto kroků provedeme nejvýše $2n$, než buď najdeme nulu, nebo matici zredukujeme na prázdnou. Jeden krok trvá konstantní čas a zabere konstantní prostor, takže celý algoritmus běží v čase $\Theta(n)$ a prostoru $\Theta(1)$.

Dodejme ještě, že asymptoticky rychleji to nejde – uvážíme-li matici, která má na vedlejší diagonále a všude nad ní samé -1 a pod diagonálou 1 . Kteroukoliv -1 na diagonále můžeme změnit na 0 , aniž bychom porušili uspořádání. Takže každý korektní algoritmus musí přechít všech n prvků diagonály.

2 Průchod spojovým seznamem (společné okruhy)

Předpokládejme 16bitový big-endian RISC procesor, který podporuje instrukce uvedené v tabulce níže. Registry procesoru jsou 16bitové, virtuální adresový prostor dovoluje adresovat právě pomocí 16bitových adres.

Procesor má k dispozici 10 obecných registrů R0 až R9, které uchovávají unsigned integer o velikosti 16 bitů (další registry nejsou pro naši úlohu potřeba; pro řešení využijte pouze registry, pomocné výsledky neukládejte do paměti). V celé otázce pracujeme jen s virtuálními adresami, překlad na fyzické adresy není potřeba vůbec řešit.

Zároveň máme k dispozici strukturu pro počítání frekvencí čísel využívající triviální spojový seznam. V celé otázce předpokládáme korektně vytvořený spojový seznam, tj. není potřeba detektovat cyklus, špatné zarovnání v paměti apod.

```
typedef struct freq freq_t;
struct freq {
    uint16_t number;
    uint16_t count;
    freq_t *next;
};
// next je u posledního prvku nastaveno na 0
...

freq_t *head;
```

RD označuje cílový registr, RS pak značí zdrojové registry. memory je pro přístup do paměti na dané adrese.

LI RD, imm	RD := imm (načtení konstanty)
LW RD, offset(RS)	RD := memory[RS + offset]
ADD RD, RS1, RS2	RD := RS1 + RS2
OR RD, RS1, RS2	RD := RS1 bitwise or RS2
SW RS1, offset(RS2)	memory[RS2 + offset] := RS1
BEZ RS, label	if RS == 0: jump to label
BNEZ RS, label	if RS != 0: jump to label
JMP label	jump to label

imm a offset jsou konstanty (nelze použít registr!)

Proměnná `head` je uložena v paměti na adrese `0x900a` (tj. v tomto místě je pointer na první skutečný prvek seznamu).

Část A: vyznačte orámováním a šipkami jednotlivé prvky seznamu v následujícím hexdumpu paměti. (Hexdump je ve stejné podobě k dispozici i na další stránce, zřetelně vyznačte, který obsahuje finální řešení.)

```
8ff0: 45 2f 90 11 8f 0f 90 42 01 23 45 90 42 42 90 90
9000: 89 e2 26 90 05 ff 00 de ad 00 90 10 90 14 00 00
9010: 00 01 00 05 90 42 ca fe ca fe 42 90 30 be ef 35
9020: 45 01 90 54 42 53 01 87 aa bb 01 23 45 67 90 24
9030: 12 02 08 90 17 44 ab cd 90 48 20 17 00 5f 00 00
9040: 77 88 01 23 00 f0 90 3a 5e d3 41 90 48 90 4c 00
9050: 39 4a 85 90 24 48 89 00 90 10 90 00 42 23 ca fe
```

Část B: doplňte posloupnost instrukcí (v pravém sloupečku) tak, aby v registru R3 bylo uloženo celkové množství prvků, tj. součet položek `count` ve výše uvedené struktuře. Okomentujte využití jednotlivých registrů (tj. jak by pravděpodobně byly pojmenované odpovídající proměnné).

Váš kód musí být napsán obecně pro libovolný platný seznam. Tj. doplněný kód by neměl obsahovat žádné pevné adresy, kromě adresy hlavy seznamu (která je již předpřipravená). Váš kód nemá řešit případné přetečení při sčítání prvků nebo kontrolu přístupu do paměti.

V případě skoků využijte pojmenovaná návěští (např. `loop_start`).

```
LI R1, 0x900a
LW R2, 0(R1)
LI R3, 0
// <-- sem doplňte vhodné instrukce
end:
// v registru R3 je součet položek count
```

(Druhá strana obsahuje tentýž instrukční blok, finální instrukce doplňte do něj.)

Student 666. Řešení můžete vyznačit na této stránce, která reprodukuje hexdump a kód ve větším formátu.

```
8ff0: 45 2f 90 11 8f 0f 90 42 01 23 45 90 42 42 90 90
9000: 89 e2 26 90 05 ff 00 de ad 00 90 10 90 14 00 00
9010: 00 01 00 05 90 42 ca fe ca fe 42 90 30 be ef 35
9020: 45 01 90 54 42 53 01 87 aa bb 01 23 45 67 90 24
9030: 12 02 08 90 17 44 ab cd 90 48 20 17 00 5f 00 00
9040: 77 88 01 23 00 f0 90 3a 5e d3 41 90 48 90 4c 00
9050: 39 4a 85 90 24 48 89 00 90 10 90 00 42 23 ca fe
```

```
8ff0: 45 2f 90 11 8f 0f 90 42 01 23 45 90 42 42 90 90
9000: 89 e2 26 90 05 ff 00 de ad 00 90 10 90 14 00 00
9010: 00 01 00 05 90 42 ca fe ca fe 42 90 30 be ef 35
9020: 45 01 90 54 42 53 01 87 aa bb 01 23 45 67 90 24
9030: 12 02 08 90 17 44 ab cd 90 48 20 17 00 5f 00 00
9040: 77 88 01 23 00 f0 90 3a 5e d3 41 90 48 90 4c 00
9050: 39 4a 85 90 24 48 89 00 90 10 90 00 42 23 ca fe
```

```
LI R1, 0x900a
LW R2, 0(R1)
LI R3, 0
```

```
end:
// v registru R3 je soucet polozek count
```

Nástin řešení V hexdumpu je na adrese 0x900a adresa prvního („skutečného“) prvku (tj. 0x9010), na adrese 0x9014 je pak adresa následujícího atd. (celkem 3 prvky, 5×1 , 240×291 a 95×8215).

U instrukcí je potřeba začít kontrolou na prázdný seznam. Registr R3 obsahuje od začátku průběžný součet, R2 je adresa aktuálního prvku (na konci seznamu bude obsahovat 0); do pomocného registru R4 ukládáme položku count právě zpracovávaného prvku.

```
LI R1, 0x900a
LW R2, 0(R1)
LI R3, 0
loop:
BEZ R2, end
LW R4, 2(R2)
ADD R3, R3, R4
LW R2, 4(R2)
JMP loop
end:
// v registru R3 je součet položek count
```

3 Lineární algebra, matice (společné okruhy)

Uvažujme matice

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 3 & 1 \\ 1 & 1 & 3 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 2 & 3 \\ 1 & 2 & 2 \\ 3 & 3 & 1 \end{pmatrix}.$$

1. Najděte bázi prostoru

$$U = \{x \in \mathbb{R}^3 : Ax = Bx\}.$$

2. Vypočítejte $\det(A^2 B A^T B^{-1})$.
3. Rozhodněte, zda je matice $A - B$ diagonalizovatelná. Rozklad hledat nemusíte, ale jeho existenci zdůvodněte.

Nástin řešení

1. Musíme najít jádro matice

$$A - B = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 1 & -1 \\ -2 & -2 & 2 \end{pmatrix}.$$

Bázi jádra tvoří např. vektory $(1, 0, 1)^T$, $(0, 1, 1)^T$.

2.
$$\det(A^2 B A^T B^{-1}) = \det(A)^2 \det(B) \det(A) \det(B)^{-1} = \det(A)^3 = (-5)^3 = -125.$$

3. Jádro $A - B$ má dimenzi 2 a stopa $A - B$ je 3. Tudíž 0 je vlastní číslo s algebraickou i geometrickou násobností 2 a zbývajícím vlastním číslem je 3 (s násobností 1). Z toho plyne existence báze $\mathbb{R}^3$ složené z vlastních vektorů $A - B$ a ta je tím pádem diagonalizovatelná.

4 Interval spolehlivosti pro střední hodnotu (společné okruhy)

1. Nechtě $X_1, \dots, X_n$ jsou nezávislá pozorování z normálního rozdělení $N(\mu, \sigma^2)$, kde σ známe, μ ne. Napište 95% interval spolehlivosti (konfidenční interval) pro μ .
2. Měříme dobu běhu programu. Předpokládejme, že doba jednoho běhu má normální rozdělení se známou směrodatnou odchylkou $\sigma = 12$ ms. Z $n = 36$ nezávislých měření vyšel průměr $\bar{X}_{36} = 105$ ms. Spočítejte 95% interval spolehlivosti pro skutečnou střední dobu běhu.

3. Vysvětlete slovně, co znamená označení 95% interval spolehlivosti.

Možná se vám budou hodit následující tabulky, kde Φ je distribuční funkce standardního normálního rozdělení $N(0, 1)$.

x	-2.0	-1.5	-1.0	-0.5	0.0	0.5	1.0	1.5	2.0
$\Phi(x)$	0.02	0.07	0.16	0.31	0.5	0.69	0.84	0.93	0.98

p	0.9	0.95	0.975	0.99	0.995
$\Phi^{-1}(p)$	1.2816	1.6449	1.96	2.3263	2.5758

Nástin řešení

1. Protože

$$\frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim N(0, 1),$$

dostaneme přibližně, respektive při splnění normálního modelu přesně, interval

$$\bar{X} \pm \Phi^{-1}(0.975) \frac{\sigma}{\sqrt{n}} = [\bar{X} - \Phi^{-1}(0.975) \frac{\sigma}{\sqrt{n}}, \bar{X} + \Phi^{-1}(0.975) \frac{\sigma}{\sqrt{n}}].$$

2. Dosadíme

$$105 \pm 1.96 \frac{12}{\sqrt{36}} = 105 \pm 1.96 \cdot 2 = 105 \pm 3.92.$$

Interval je tedy $[101.08, 108.92]$ ms, nebo po zaokrouhlení $[101, 109]$ ms.

3. Parametr μ chápeme jako pevnou neznámou hodnotu a interval jako náhodný, protože závisí na náhodném vzorku. Kdybychom stejný postup opakovali mnohokrát, přibližně 95% takto získaných intervalů by skutečnou hodnotu μ obsahovalo.

5 Indexace v prostorových databázích (specializace WDOP)

- Definujte R-strom a vysvětlete, k čemu se používá. Popište strukturu vnitřních a listových uzlů a vysvětlete roli minimálních ohraničujících obdélníků, tj. MBR.
- Vysvětlete na jednoduchém příkladu, jak se pomocí R-stromu vyhodnocuje dotaz "Najdi všechny objekty, které leží v zadaném obdélníku nebo ho protínají." Uveďte, kdy lze celý podstrom bezpečně vynechat, a kdy je naopak nutné pokračovat do více větví stromu.
- Porovnejte R-strom s alespoň jednou další metodou prostorového indexování, například Quad-tree, k-d-tree, Z-křivkou nebo Hilbertovou křivkou. Uveďte hlavní výhodu a nevýhodu obou přístupů.

Nástin řešení 1. R-strom je výškově vyvážený vícecestný stromový index pro prostorová data. Je zobecněním B-stromu pro vícerozměrné objekty. Každý uzel obsahuje položky ve tvaru: MBR + odkaz.

- MBR (minimum bounding rectangle) je nejmenší osově zarovnaný obdélník, který ohraničuje daný objekt nebo všechny objekty v příslušném podstromu.

- V listových uzlech jsou uloženy MBR konkrétních prostorových objektů a odkazy na tyto objekty. Ve vnitřních uzlech jsou uloženy MBR potomků a odkazy na odpovídající podstromy.

2. Při vyhodnocení takového dotazu se strom prochází od kořene. Pokud se MBR daného podstromu se zadaným obdélníkem neprotíná, celý podstrom lze přeskóčit. Pokud se protíná, pokračuje se níže. V listech se pak ověří konkrétní objekty.

3. Výhodou R-stromu je, že pracuje přímo s prostorovými objekty a hodí se i pro obecnější tvary. Nevýhodou je, že MBR různých větví se mohou překrývat, takže při dotazu může být nutné procházet více částí stromu.

Z-křivka je jiný způsob prostorového indexování. Převádí vícerozměrné souřadnice na jednu hodnotu, například tak, že se střídají bity souřadnic x a y . Nad touto jednorozměrnou hodnotou pak lze použít běžný index, například B+ strom.

Výhodou Z-křivky je jednoduchost a možnost využít klasické indexy. Nevýhodou je, že prostorová blízkost se zachovává jen přibližně. Dotaz na objekty v zadané oblasti se proto často musí rozložit na více intervalů v Z-pořadí a nalezené kandidáty je nutné dodatečně ověřit.

6 MapReduce (specializace WDOP)

Uvažujte kolekci textových dokumentů. Každý dokument má identifikátor `doc_id` a textový obsah. Cílem je vytvořit invertovaný index, tedy pro každé slovo zjistit, ve kterých dokumentech se vyskytuje a kolikrát.

Například z dokumentů:

D1: data data index D2: web data D3: index web web

má vzniknout:

data -> [(D1, 2), (D2, 1)] index -> [(D1, 1), (D3, 1)] web -> [(D2, 1), (D3, 2)]

1. Navrhněte řešení pomocí MapReduce. Uveďte, zda lze úlohu vyřešit jedním MapReduce krokem, nebo zda je vhodné použít dva kroky. Své rozhodnutí zdůvodněte.
2. Specifikujte alespoň v pseudokódu, co bude dělat funkce `map` a co bude dělat funkce `reduce`. Uveďte příklad emitovaných dvojic klíč–hodnota.
3. Vysvětlete, zda by zde pomohla funkce `combine` a k čemu obecně slouží.

Nástin řešení 1. Úlohu lze vyřešit jedním MapReduce krokem. Mapper pro každé slovo v dokumentu vypíše jako klíč dané slovo a jako hodnotu identifikátor dokumentu. Reducer potom pro každé slovo dostane seznam dokumentů, ve kterých se slovo objevilo, a spočítá četnosti výskytů podle jednotlivých dokumentů. Použití dvou kroků není nutné, ale může být užitečné u velkých dat, pokud chceme nejprve zmenšit mezivýsledky, například spočítáním lokálních četností slov v dokumentech.

2. Funkce `map` zpracuje jeden dokument. Pro každé slovo v textu vytvoří dvojici, kde klíčem je slovo a hodnotou je identifikátor dokumentu: `map(doc_id, text):` pro každé slovo v textu: `emit(slovo, doc_id)`

Například z dokumentu D1: data data index vzniknou dvojice data -> D1, data -> D1 a index -> D1.

Funkce `reduce` dostane pro každé slovo všechny hodnoty, tedy seznam dokumentů, ve kterých se slovo vyskytlo. V tomto seznamu spočítá, kolikrát se opakuje každý dokument:

`reduce(slovo, seznam_doc_id):` spočítej četnosti jednotlivých `doc_id` `emit(slovo, seznam_dvojic (doc_id, četnost))`

Například ze vstupu data -> [D1, D1, D2] vytvoří výstup data -> [(D1, 2), (D2, 1)].

3. Funkce `combine` by zde pomohla. Obecně slouží jako lokální redukce provedená po mapperu ještě před přesunem dat k reducerům. Jejím cílem je zmenšit objem mezivýsledků posílaných po síti.

V této úloze může combiner lokálně sloučit opakované výskyty stejného slova ve stejném dokumentu. Například místo tří dvojic data -> D1, data -> D1, data -> D1 může poslat kompaktnější informaci data -> (D1, 3). Reducer pak z těchto částečných výsledků sestaví konečný seznam dokumentů a četností. Combiner nemění význam výpočtu, pouze snižuje množství přenášených dat.

7 Transakční rozvrhy (specializace WDOP)

Pro následující transakční rozvrh, kde R znamená operaci čtení a W operaci zápisu a ??? reprezentuje jednu neznámou databázovou operaci čtení nebo zápisu:

	T_1	T_2	T_3
1	W(<i>Project</i>)		
2		???	
3		W(<i>Project</i>)	
4		COMMIT	
5			R(<i>Customer</i>)
6			W(<i>Project</i>)
7			COMMIT
8	R(<i>Customer</i>)		
9	COMMIT		

1. Určete všechny databázové operace čtení nebo zápisu proměnné, které po dosazení do transakce T_2 místo ??? v čase 2 způsobí, že rozvrh nebude konfliktově uspořádatelný (conflict-serializable). Své rozhodnutí zdůvodněte.
2. Určete všechny databázové operace čtení nebo zápisu proměnné, které po dosazení do transakce T_2 místo ??? v čase 2 způsobí, že rozvrh nebude zotavitelný (recoverable). Své rozhodnutí zdůvodněte.

Nástin řešení Rozvrh bez operace ??? v čase 2 je jak konfliktově uspořádatelný, tak zotavitelný. V precedenčním grafu existují pouze hrany $T_1 \rightarrow T_2$ a $T_2 \rightarrow T_3$. Je tedy acyklický. Není zde ani žádné čtení nepotvrzených dat, které by spolu se špatným pořadím operací COMMIT mohlo způsobit nezotavitelnost.

1. Pouze operace W(*Customer*). V precedenčním grafu tak přibude hrana $T_2 \rightarrow T_1$ a vznikne cyklus s existující hranou $T_1 \rightarrow T_2$. Jiné operace R(*Customer*), W(*Project*) ani R(*Project*) novou hranu nepřidají. Stejně tak DB operace na jakékoli jiné proměnné.
2. Pouze operace R(*Project*). Vznikne tím nepotvrzené čtení hodnoty *Project*, zapsané transakcí T_1 , přičemž COMMIT transakce T_2 předchází operaci COMMIT transakce T_1 . Jiné operace W(*Project*), W(*Customer*) ani R(*Customer*) čtení nepotvrzených dat nezpůsobí. Stejně tak DB operace na jakémkoliv jiné proměnné.

8 Konceptuální modelování (specializace WDOP)

V rámci návrhu datového modelu aplikace byly na konceptuální úrovni identifikovány dvě datové třídy - *Hudebník* a *Nástroj*. O hudebníkovi je nutné si pamatovat jeho unikátní kombinaci jména, příjmení a roku narození. O nástroji je potřeba si pamatovat jeho unikátní název, a dále typ. Dále byly identifikovány následující vztahy:

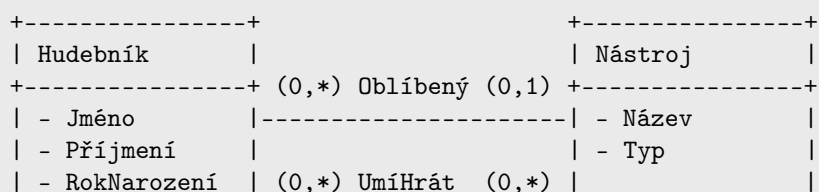
- Hudebník má nejvýše jeden *oblíbený* nástroj. Jeden nástroj může být oblíbený libovolným počtem hudebníků.
- Hudebník může *umět hrát* na libovolný počet nástrojů a naopak na jeden nástroj může umět hrát libovolný počet hudebníků. Je potřeba vědět, od kterého roku hudebník na nástroj hraje.

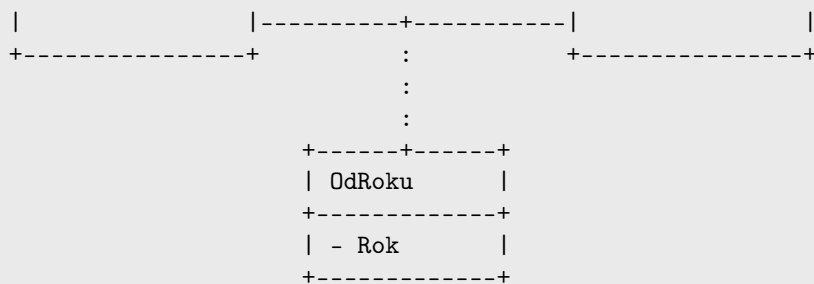
Pro výše popsanou situaci

1. Načrtněte pro popsanou situaci odpovídající UML nebo ER model. Nezapomeňte vyznačit kardinality vztahů.
2. Převeďte Vámi navržený konceptuální model na logický relační model. Nezapomeňte vyznačit všechny klíče a referenční integritu (cizí klíče). Převod proveďte tak, aby v cizích klíčích nemusela být nikdy vložena NULL hodnota.
3. Je logický relační model vzniklý převodem konceptuálního modelu vhodný z hlediska dosažené normální formy? Pokud ne, jak by měl být logický relační model upraven, aby vhodný byl a proč?

Nástin řešení

1. Řešení v UML:





2.
 - Hudebník(HudebníkID, Jméno, Příjmení, RokNarození)
 - Nástroj(NástrojID, Nazev, Typ)
 - Oblíbený(HudebníkID, NástrojID),
 $\text{HudebníkID} \subseteq \text{Hudebník.HudebníkID}$,
 $\text{NástrojID} \subseteq \text{Nástroj.NástrojID}$
 - UmíHrát(HudebníkID, NástrojID, Rok),
 $\text{HudebníkID} \subseteq \text{Hudebník.HudebníkID}$,
 $\text{NástrojID} \subseteq \text{Nástroj.NástrojID}$

Kardinalita vztahů se projeví v rozdílné definici klíčů vztahových tabulek.

3. Jediné funkční závislosti jsou ty na klíčích, a tedy je výsledné schéma v BCNF. Není potřeba nic dalšího dělat.

9 Lineární rekurence (specializace OI-G-O, OI-G-PADS, OI-G-PDM, OI-O-PADS, OI-O-PDM, OI-PADS-PDM)

- (a) Necht' $a_0, a_1, a_2, \dots$ je nekonečná posloupnost reálných čísel taková, že $a_0 = -1$, $a_1 = 0$ a $a_{n+2} = 5a_{n+1} - 6a_n$ platí pro každé nezáporné celé číslo n . Určete explicitní vzorec pro a_n .
- (b) Necht' $b_0, b_1, b_2, \dots$ je nekonečná posloupnost reálných čísel taková, že $b_n = 2^n + (-3)^n$ pro každé nezáporné celé číslo n . Nalezněte lineární rekurenci (obdobnou té v podúloze (a)), kterou tato posloupnost splňuje.

Nástin řešení

- (a) Například pomocí vytvářejících funkcí: Necht' $A(x) = \sum_{n=0}^{\infty} a_n x^n$. Pak z dané rekurence plyne $A(x) - 5xA(x) + 6x^2A(x) = a_0 + (a_1 - 5a_0)x = -1 + 5x$, a tedy

$$\begin{aligned}
 A(x) &= \frac{5x - 1}{1 - 5x + 6x^2} = \frac{5x - 1}{(1 - 2x)(1 - 3x)} \\
 &= \frac{-3}{1 - 2x} + \frac{2}{1 - 3x} = -3 \cdot \sum_{n=0}^{\infty} (2x)^n + 2 \cdot \sum_{n=0}^{\infty} (3x)^n \\
 &= \sum_{n=0}^{\infty} (-3 \cdot 2^n + 2 \cdot 3^n) x^n.
 \end{aligned}$$

Srovnáním s definicí $A(x)$ dostáváme

$$a_n = -3 \cdot 2^n + 2 \cdot 3^n.$$

Existují i jiné metody řešení, např. očekáváme-li pro rekurenci $a_{n+2} = 5a_{n+1} - 6a_n$ (bez předem určených hodnot prvních dvou členů) řešení ve tvaru $a_n = c^n$, snadno nalezneme dvě bázecká řešení $a_n = 2^n$ a $a_n = 3^n$, nahlédneme, že obecné řešení musí být jejich lineární kombinací, a dle prvních dvou členů dopočítáme koeficienty.

- (b) Například obrácením výše uvedeného postupu: Necht' $B(x) = \sum_{n=0}^{\infty} b_n x^n$. Z explicitního vzorce pro koeficienty dostáváme

$$B(x) = \frac{1}{1 - 2x} + \frac{1}{1 + 3x} = \frac{2 + x}{1 + x - 6x^2},$$

a tedy

$$B(x) + xB(x) - 6x^2B(x) = 2 + x.$$

Speciálně tedy pro každé nezáporné celé číslo n se musí vyrovnat koeficienty u x^{n+2} na levé straně, a tedy

$$b_{n+2} + b_{n+1} - 6b_n = 0,$$

či jinak řečeno

$$b_{n+2} = 6b_n - b_{n+1}.$$

10 Použití LP v algoritmech (specializace OI-G-O, OI-O-PADS, OI-O-PDM)

1. Formulujte přesné znění Farkasova lemmatu.
2. Je dán neorientovaný graf $G = (V, E)$ a vrcholy $s, t \in V$. Označme P množinu všech cest mezi vrcholy s a t v daném grafu. Uvažme následující lineární program, ve kterém máme proměnnou $x_e \in E$ pro každou hranu grafu G :

$$\begin{aligned} \min \quad & \sum_{e \in E} x_e \\ \sum_{e \in p} x_e & \geq 1 \quad \forall p \in P \\ x_e & \geq 0 \quad \forall e \in E \end{aligned}$$

Napište duální program (duální proměnné označujte y , s patřičnými indexy).

3. Nechť $x \in \mathbb{R}_{\geq 0}^E$ je optimální řešení lineárního programu z předešlého bodu a pro každý vrchol $u \in V$, nechť $d(u)$ označuje délku nejkratší cesty mezi vrcholy s a u vzhledem k délkám hran daným řešením x (tj., délka hrany e je x_e).

Uvažme funkci $f : (0, 1) \rightarrow 2^E$ definovanou takto: Pro každé $r \in (0, 1)$,

$$f(r) = \{uv \in E \mid d(u) \leq r \text{ a } r < d(v)\},$$

kde pro stručnost zápisu používáme pro hranu mezi vrcholy u a v označení uv .

Rozhodněte, zda pro některé hodnoty $r \in (0, 1)$ platí, že množina $f(r)$ je $s - t$ řez.

Rozhodněte, zda pro některé hodnoty $r \in (0, 1)$ platí, že množina $f(r)$ je nejmenší $s - t$ řez.

Nástin řešení

1. **Věta (Farkasovo lemma).** Soustava nerovnic $Ax \leq b$ má řešení, právě když neexistuje nezáporné $y \in \mathbb{R}^m$ takové, že $y^T A = 0$ a $y^T b < 0$.
2. V duálním LP je proměnná y_p pro každou cestu $p \in P$. Účelová funkce a podmínky jsou následující:

$$\begin{aligned} \max \quad & \sum_{p \in P} y_p \\ \sum_{p: e \in p} y_p & \leq 1 \quad \forall e \in E \\ y_p & \geq 0 \quad \forall p \in P \end{aligned}$$

3. Uvažme libovolné $r \in (0, 1)$. Pro spor předpokládejme, že množina hran $F = f(r)$ není řezem mezi vrcholy s a t . Existuje tudíž nějaká cesta p mezi s a t ; označme k její délku a $v_0 = s, v_1, \dots, v_k = t$ její vrcholy. Nechť $i = \arg \min_i \{d(v_i) > r\}$. Protože $d(s) = 0, d(t) = 1$, a $r \in (0, 1)$, takové i jistě existuje, a navíc splňuje $i > 0$. Pak ovšem hrana $u_{i-1}u_i$ patří do množiny F , což je spor s tím, že p je cestou v grafu $(V, E \setminus F)$. Proto pro každé $r \in (0, 1)$, množina $f(r)$ je $s - t$ řezem.

Nechť $OPT = \sum_{e \in E} x_e$ (tj. OPT je optimální hodnota účelové funkce), a nechť C je velikost nejmenšího $s - t$ řezu v grafu G . Protože náš lineární program je relaxací problému nejmenšího $s - t$ řezu, platí $C \geq OPT$. Protože, jak jsme nahlédli, $f(r)$ je $s - t$ řezem pro každé $r \in (0, 1)$, platí také $|f(r)| \geq C$, pro každé $r \in (0, 1)$.

Uspořádejme vrcholy grafu podle hodnoty $d(\cdot)$: $0 = d(v_0) \leq d(v_1) \leq \dots d(v_n) = 1$. Pak platí:

$$C \geq OPT = \sum_{e \in E} x_e \geq \sum_{uv \in E} |d(u) - d(v)| = \sum_{i=1}^{n-1} (d(v_{i+1}) - d(v_i)) \cdot |f(d(v_i))| \geq C \cdot \sum_{i=1}^{n-1} (d(v_{i+1}) - d(v_i)) = C,$$

tudíž všechny nerovnosti platí jako rovnosti. Proto pro každé $i \in \{1, \dots, n-1\}$, $|f(d(v_i))| = C$, což dále implikuje $|f(r)| = C$ pro každé $r \in (0, 1)$. Neboli pro každé $r \in (0, 1)$ je $f(r)$ nejmenším $s-t$ -řezem.

11 Voroného diagram (specializace OI-G-O, OI-G-PADS, OI-G-PDM)

1. Definujte region Voroného diagramu pro konečnou množinu M v $\mathbb{R}^d$.
2. Kolik nejvíce vrcholů (asymptoticky) může mít jeden region Voroného diagramu n bodů v $\mathbb{R}^4$? Pokuste se dokázat horní odhad i nalézt konstrukci vhodné n -prvkové množiny pro dolní odhad. (Není nutné uvádět přesné souřadnice bodů, stačí postup konstrukce.)

Nástin řešení

1. Jiří Matoušek, Introduction to Discrete Geometry, Section 5.6.
2. Z definice vyplývá jednoduchý důsledek [Jiří Matoušek, Introduction to Discrete Geometry, Observation 5.6.1], že region je konvexní mnohostěn s $n-1$ fasetami. Průnikem R s 5 uzavřenými poloprostory definujícími obrovský simplex dostaneme konvexní mnohostěn s nejvýše $n+4$ fasetami a aspoň tolik vrcholy jako v R . Podle upper bound věty [Jiří Matoušek, Introduction to Discrete Geometry, Theorem 5.5.2] použité na duální mnohostěn má tedy region nejvýše $O(n^2)$ vrcholů. Pro konstrukci můžeme vzít duální mnohostěn D k $(n-1)$ -vrcholovému cyklickému mnohostěnu, umístit jeden bod p dovnitř D , a zbylých $n-1$ bodů definovat jako zrcadlové obrazy p podle nadrovin definujících fasety D .

12 Mohutnost množin (specializace OI-G-PDM, OI-O-PDM, OI-PADS-PDM)

- Napište definici pojmu „množiny X a Y mají stejnou mohutnost“.
- Ukažte, že následující množiny mají stejnou mohutnost (pokud budete v důkazu chtít použít Cantor-Bernsteinovu větu, zformulujte ji, ale nemusíte ji dokazovat):
 - $\mathbb{R}$ (množina všech reálných čísel)
 - $\{0, 1\}^{\mathbb{N}}$ (množina všech nekonečných posloupností nul a jedniček)
 - $\mathbb{R}^2$ (množina všech dvojic reálných čísel)
- Uveďte příklad množiny, která má ostře menší mohutnost než $\mathbb{R}$, a příklad množiny, která má ostře větší mohutnost než $\mathbb{R}$ (stačí bez důkazu).

Nástin řešení

- Množiny X a Y mají stejnou mohutnost, jestliže existuje bijekce $X \rightarrow Y$.
- K důkazu použijeme Cantor-Bernsteina větu, která říká, že pokud pro množiny X a Y existuje prostá funkce $X \rightarrow Y$ a prostá funkce $Y \rightarrow X$, pak X a Y mají stejnou mohutnost.

Každé reálné číslo x můžeme vyjádřit ve dvojkové soustavě jako $\pm b_m b_{m-1} \dots b_0, b_{-1} b_{-2} \dots$, kde b_i jsou 0 či 1 (toto vyjádření není jednoznačné, nicméně si pro každé reálné číslo stačí kanonicky zvolit jednu reprezentaci, například tu nekončící periodou 1). Prostou funkci z $\mathbb{R}$ do $\{0, 1\}^{\mathbb{N}}$ pak získáme například tak, že číslu x přiřadíme posloupnost

$$p, 0, b_m, 0, b_{m-1}, \dots, 0, b_0, 1, b_{-1}, 1, b_{-2}, \dots,$$

kde $p = 1$ je-li číslo x nezáporné a $p = 0$ je-li záporné.

Oproti tomu prostou funkcí z $\{0, 1\}^{\mathbb{N}}$ do $\mathbb{R}$ získáme například tak, že posloupnosti $a_1, a_2, \dots$ přiřadíme reálné číslo, jehož zápis v trojkové soustavě je $0, a_1 a_2 \dots$ (vyjádření ve dvojkové soustavě by nefungovalo, jelikož tím bychom například posloupností $1, 0, 0 \dots$ a $0, 1, 1, \dots$ přiřadili stejné číslo $1/2$).

Jelikož $\mathbb{R}$ a $\{0, 1\}^{\mathbb{N}}$ mají stejnou mohutnost, abychom ukázali, že $\mathbb{R}$ a $\mathbb{R}^2$ mají stejnou mohutnost, stačí ukázat, že $\{0, 1\}^{\mathbb{N}}$ a $(\{0, 1\}^{\mathbb{N}})^2$ mají stejnou mohutnost (jsou-li $f: \mathbb{R} \rightarrow \{0, 1\}^{\mathbb{N}}$ a $g: (\{0, 1\}^{\mathbb{N}})^2 \rightarrow \{0, 1\}^{\mathbb{N}}$ bijekce, pak funkce $(x, y) \mapsto f^{-1}(g(f(x), f(y)))$ je bijekce z $\mathbb{R}^2$ do $\mathbb{R}$). Zde můžeme zkonstruovat přímo bijekci, posloupnosti $a_1, a_2, a_3, a_4, \dots$ přiřadíme dvojici posloupností $a_1, a_3, a_5, \dots$ a $a_2, a_4, a_6, \dots$.

- Ostře menší mohutnost má například množina přirozených čísel nebo libovolná konečná množina. Ostře větší mohutnost má například potenční množina $2^{\mathbb{R}}$.

13 Toky v sítích (specializace OI-G-PADS, OI-O-PADS, OI-PADS-PDM)

Uvažme bipartitní graf se stejně velkými partitami A a B a množinou hran E . *Perfektní párování* říkáme množině hran $M \subseteq E$ takové, že každý vrchol je incidentní s právě jednou hranou z M . *Cyklové pokrytí* je množina kružnic v grafu taková, že každý vrchol grafu leží na právě jedné z nich.

1. Ukažte, jak hledání perfektního párování v bipartitním grafu převést na hledání maximálního celočíselného toku ve vhodné síti. Dokažte, že převod funguje.
2. Vyberte vhodný algoritmus na hledání maximálního celočíselného toku v síti z předchozího bodu a analyzujte jeho složitost vzhledem k počtu vrcholů n a počtu hran m zadaného bipartitního grafu. Pokuste se využít omezeného rozsahu kapacit.
3. Podobným způsobem vyřešte úlohu hledání cyklového pokrytí bipartitního grafu.

Nástin řešení

1. Vytvoříme síť s jednotkovými kapacitami. K vrcholům z partit A a B přidáme ještě zdroj z a stok s . K hranám původního grafu (orientovaným z A do B) přidáme ještě hrany ze z do všech vrcholů A a ze všech vrcholů B do s .

Ukážeme, že ke každému perfektnímu párování existuje tok velikosti $|A|$. Pro každou hranu $\{a, b\} \in M$ ($a \in A, b \in B$) necháme téci 1 po hranách sítě (z, a) , (a, b) , (b, s) . Jelikož M je párování, žádnou hranu nevyužijeme vícekrát.

Naopak je-li v síti celočíselný tok velikosti $|A| = |B|$, existuje stejně velké párování. Stačí vzít hrany z A do B , po kterých teče 1. Tyto hrany musí tvořit párování: pokud by nějaké dvě sdílely vrchol $a \in A$, z tohoto vrcholu by odtékaly aspoň 2 jednotky, ale přitéci může nejvýše 1, což je spor s Kirchhoffovým zákonem. Analogicky pokud by dvě hrany sdílely $b \in B$, do tohoto vrcholu by přitékaly aspoň 2 jednotky, ale odtéci může nejvýš 1.

2. Stačí použít Fordův-Fulkersonův algoritmus. Označíme-li počet vrcholů a hran sítě N a M , každá iterace algoritmu spotřebuje čas $\mathcal{O}(M)$ na nalezení a zpracování jedné zlepšující cesty. Jelikož kapacity jsou jednotkové, jedna iterace zvýší velikost toku o alespoň 1, takže počet iterací je shora omezen velikostí maximálního toku, a ta nepřekročí N . F.-F. algoritmus tedy doběhne v čase $\mathcal{O}(NM)$. Zbývá dosadit $N = n + 2$ a $M = m + 2n$ a získáme celkovou složitost $\mathcal{O}(nm)$. Do ní se vejde i přeložení grafu na síť a toku na párování.
3. Cyklové pokrytí je ekvivalentní s podmnožinou hran takovou, že každý vrchol inciduje s právě dvěma jejími hranami. Síť zkonstruujeme stejně až na to, že hrany ze zdroje a do spotřebiče dostanou kapacitu 2. Rozbor složitosti povedeme analogicky a vyjde $\mathcal{O}(nm)$.

14 Množiny a funkce ve dvou dimenzích (specializace OI-G-O, OI-G-PADS, OI-G-PDM, OI-O-PADS, OI-O-PDM, OI-PADS-PDM)

V této úloze vám může pomoci vzorec $\sin(\pi/6) = \sin(5\pi/6) = 1/2$.

1. Uvažujme následující podmnožiny $\mathbb{R}^2$:

$$A = \{(x, y) \in \mathbb{R}^2; y \leq \sin(x)\}$$

$$B = A \cap \{(x, y) \in \mathbb{R}^2; y \geq \frac{1}{2}\}$$

$$C = B \cap \{(x, y) \in \mathbb{R}^2; 0 < x < \pi\}.$$

Které z množin A , B , C jsou uzavřené? Které z nich jsou kompaktní?

2. Pro množinu B z předchozí podotázky rozhodněte, zda existuje spojitá funkce $f: B \rightarrow \mathbb{R}$, která je na množině B zdola omezená, ale nenabývá na množině B svého minima.
3. Spočítejte obsah množiny C .

Nástin řešení

1. Všechny tři množiny jsou uzavřené. U A to plyne třeba z toho, že A je vzor uzavřené množiny $[0, +\infty)$ vůči spojitě funkci $f(x, y) = \sin(x) - y$. U B z toho, že průnik dvou uzavřených množin je uzavřený, a podobně C lze ekvivalentně popsat jako průnik $B \cap \{(x, y) \in \mathbb{R}^2; \pi/6 \leq x \leq 5\pi/6\}$. Kompaktní je pouze množina C , protože A ani B nejsou omezené.
2. Taková spojitá funkce existuje: například $f(x, y) = \exp(-x^2)$ je kladná, a tedy zdola omezená, a na B nabývá libovolně malých kladných hodnot, ale nenabývá na B minima.
3. Obsah spočítáme obvyklým způsobem jako integrál:

$$\int_{\pi/6}^{5\pi/6} \left(\sin(x) - \frac{1}{2} \right) dx = \left[-\cos(x) - \frac{x}{2} \right]_{\pi/6}^{5\pi/6} = \sqrt{3} - \frac{\pi}{3}.$$

15 Anti-aliasing (specializace PGVVH-PG, PGVVH-VPH, PGVVH-PV)

Anti-aliasing je technika, která může významně vylepšit vzhled rastrových grafických výstupů.

1. Který z parametrů rastrového zobrazovacího zařízení je anti-aliasingem vylepšen (obrázek se nám jeví jako na „kvalitnějším“ zařízení – ale v jakém smyslu)? Jak se to pozná na výsledném obrázku? Jak byste se u výstupu do okna ve Windows přesvědčili, zda byl anti-aliasing použit?
2. Jak se může anti-aliasing implementovat v klasickém rastrovém vykreslování (rasterizace – např. při kreslení vektorové grafiky)?
3. Jak se anti-aliasing implementuje v prostředí paprskového zobrazovače (např. Ray-tracing)?

Nástin řešení

1. Anti-aliasing (vyhlazení) je vylepšení vzhledu nakresleného objektu do rastrového výstupního zařízení. Okraje vypadají více hladké, textury se v dálce „nezrní“ (potlačení interference, Moiré efektu). Je to vlastně imitace vyššího rozlišení displeje za pomoci barevných přechodových odstínů v okrajových pixelech. Virtuálně se zvyšuje rozlišení displeje.

Matematický model pixelu: čtvereček (plocha!)

Barva pixelu: integrální průměr barvy na ploše toho čtverečku. Tj. např. když se kreslí vybarvený kruh, tak na jeho okraji jsou pixely ne zcela pokryté kruhem, ty je potřeba vybarvit barvou tak sytou, jako je podíl zakryté plochy.

Přesněji: je-li α podíl zakrytí pixelu objektem, je výsledná barva: $\alpha \cdot \text{barvaObjektu} + (1 - \alpha) \cdot \text{barvaPozadí}$.

Jak se o A-A přesvědčit ve Windows: pořídit si screenshot daného okna a potom si tento rastrový obrázek prohlédnout se zvětšením (je lepší mít prohlížeč obrázků nastavený tak, aby sám neprováděl žádná vylepšení, žádné interpolace) - pokud budou na hranách vidět přechodové barevné odstíny, byl použit anti-aliasing.

2. Při rastrovém vykreslování (např. v interpretu SVG) se dá představit cílový pixel jako čtvereček $M \times M$ subpixelů (např. 4×4), kreslit vše původními algoritmy do obrázku s 4×4 vyšším rozlišením a potom výsledek zprůměrovat (filtrovat) do původního požadovaného rozlišení. Umí to dělat (nebo je to snadno implementovatelné na) GPU.

3. V Ray-tracingu se do jednoho pixelu posílá více paprsků místo jednoho. Ve 2D modelu pixelu (jednotkový čtvereček na ploše virtuálního senzoru) se musí použít některá z vzorkovacích metod (sampling), např. Jittering, Hammersley nebo “N-rooks”. V případě potřeby (vyšší efektivita) lze implementovat i adaptivní převzorkování, kde se více vzorků (paprsků) posílá jen tam, kde je to potřeba: kde je obrazová funkce “zajímavá”, to se zjistí primárním hrubším vzorkováním.

Jittering: rozdělím čtvereček pixelu na $M \times M$ podčtverečků a do každého umístím jeden vzorek jako nezávislý náhodný pokus. Je to nestranný odhad požadovaného integrálu a potlačuje interference i vytváření shluků vzorků (šum).

16 Homogenní souřadnice a maticové transformace (specializace PGVVH-PG, PGVVH-VPH, PGVVH-PV)

1. Jak se pomocí matic transformují objekty v 3D počítačové grafice?
2. Proč zavádíme homogenní souřadnice a matice 4×4 ? Co nám umožňují realizovat?
3. Uveďte několik elementárních maticových transformací (translace, rotace, škálování – pokuste se matice napsat prvek po prvku) a jeden praktický příklad složené transformace (nemusíte konstrukci dotáhnout do konce, stačí naznačit postup).

Nástin řešení 1. Objekty jsou obvykle definovány svými vrcholy nebo jinými významnými body. Všechny tyto body se podrobují maticovým transformacím. Je žádoucí, aby transformační systém uměl všechny běžné operace (translace, otočení, škálování...) a aby se daly složitější transformace sestavovat z těch jednodušších. To všechno splňují maticové transformace maticemi 3×3 (homomorfismus = bez translace) nebo 4×4 (homogenní matice transformace, která umí i translaci).

2. Umožňují translaci (posunutí) a dále projektivní (ne-afinní) transformace. Všechny tyto věci umí mnoho let i grafický HW (GPU karty). Bylo by vhodné ukázat normální i homogenní matici (viz každá učebnice).

3. Elementární transformační matice jsou v každé učebnici geometrie.

Příklad praktické složené transformace ve 3D: potřebujeme otočit těleso kolem jeho středu C maticí rotace R (ta může být třeba jen 3×3). Řešení – použijeme dvě translace a danou rotaci, výsledná matice bude mít rozměr 4×4 . Celková matice bude $T = Tr(C) \cdot R \cdot Tr(-C)$

17 Rekurzivní sledování paprsku (specializace PGVVH-PG, PGVVH-VPH, PGVVH-PV)

Budeme se zabývat algoritmem zobrazování 3D scény, který se nazývá „Ray tracing“ (RT, česky by to bylo „Rekurzivní sledování paprsku“). Vaše slovní odpovědi můžete doprovodit obrázky a schémata, ale nezapomeňte je opatřit vysvětlivkami.

1. Popište princip algoritmu rekurzivního sledování paprsku
2. Z jakých složek se počítá světlo v rekurzivní funkci `shade()`? U jednotlivých složek můžete uvést, do jaké míry jsou fyzikálně správné (škála může být „přesně takto to v přírodě funguje“ – „ok, ale vzorec je trochu zjednodušen“ – „neodpovídá to fyzikální realitě, je tam jen kvalitativní shoda“).
3. Jaké hlavní nedostatky vykazuje základní algoritmus RT popsáný v prvním bodě, pokud bychom ho chtěli použít jako fotorealistickou zobrazovací metodu? Zkuste uvést aspoň rámcově, jak se dají jednotlivé nedostatky potlačit či omezit.

Nástin řešení Každým pixelem virtuálního rastrového obrázku (obrazovky) se pošle paprsek proti směru šíření světla do 3D scény (to navrhl již 1984 Whitted). Paprsek se protne se scénou a pokud na něco narazí, přenesení se do pixelu barva toho průsečíku na povrchu tělesa. Pokud není zasaženo žádné těleso, vezme se barva pozadí. Sondovací paprsek se nejčastěji implementuje jako rekurzivní funkce

```
RGBcolor shade(Point3D origin, Vector3D direction, int recursionDepth)
```

Uvnitř funkce `shade()` se zkombinují tyto složky:

1. Přímé osvětlení povrchu tělesa ze všech definovaných světelných zdrojů. U každého zdroje světla se nejprve zkontroluje, zda má přímou viditelnost na průsečík (pokud ne, zdroj se ignoruje). Pak se aplikuje některý z lokálních modelů odrazu světla, například Phong. Přesnost: závisí na přesnosti modelu odrazu světla, ten může být hodně věrný. Ostré stíny se přepočítávající přesně.
2. Pokud inkrementovaná hloubka rekurze dosáhla limitu, žádné další složky se nepočítají a dosavadní hodnota se rovnou vrátí. Přesnost: zde se dopouštíme chyby, protože redukuje množství světla. Obrázek bude tmavší než by měl být.
3. U potenciálně lesklého povrchu se spočítá zrcadlově odražený vektor a funkce `shade()` se zavolá rekurzivně. Výsledek se přičte přenásobený vhodnou konstantou (“lesklost” povrchu). Přesnost: největší chybou je předpoklad zrcadlového směru odrazu paprsku. Reálné materiály takhle nefungují.
4. U průhledného materiálu se podle indexů lomu určí zalomený směr paprsku a také se funkce `shade()` zavolá rekurzivně. Výsledek se přičte přenásobený vhodnou konstantou (“průhlednost” povrchu). Přesnost: chybou je opět předpoklad dokonale rovného povrchu, na kterém se světlo láme.

Akumulovaná barva se z funkce `shade()` vrátí jako výsledek. Má mít sémantiku “co by viděl pozorovatel z budou `'origin'`, kdyby se díval směrem `'direction'`”.

Hlavní nedostatky:

1. Rekurse je omezována, obrázek je tedy tmavší (existuje i korektnější omezování rekurze, tzv. Ruská ruleta).
2. Zrcadlový směr odrazu/lomu. Mělo by se použít větší množství odražených/zalomených paprsků (Monte-Carlo odhad skutečné funkce odrazivosti povrchu).
3. Nepřímé osvětlení – RT umí jen přímé osvětlení od viditelných zdrojů světla, to hrubě neodpovídá realitě. Měly by se započítat i delší cesty světla od zdrojů k průsečíku, to ale základní R-T neumí.
4. Bodové nebo směrové světelné zdroje (u kterých lze jednoduše spočítat viditelnost z průsečíku) jsou nerealistické. V přírodě existují jen plošné zdroje světla, ty by se měly zohlednit tím, že by se zavedla částečná viditelnost zdroje.
5. Jeden paprsek na pixel je málo, je třeba použít anti-aliasing (supersampling – opět Monte-Carlo) a vzorkovat barvu pixelu mnoha primárními paprsky.

18 Textury ve 3D grafice (specializace PGVVH-PG)

1. Jaký je rozdíl mezi 2D a 3D texturou? Uveďte typické aplikace těchto dvou přístupů.
2. Jaké jsou hlavní výhody a nevýhody 3D textur při použití v ray-tracingu?
3. Jak je možné napodobit vnitřní strukturu materiálu (dřevo, mramor) pomocí textury nebo procedurální definice v paprskovém zobrazovači (ray-tracing, path-tracing)? Pište o simulaci struktury materiálu, ne o zobrazovači.

Nástin řešení 1. 2D textura je plošný rastrový obrázek nebo předpis, který se na povrch 3D objektu musí mapovat (mapování textury, texturové souřadnice $[u, v]$ nebo $[s, t]$). Mapování může dělat problémy v případě analyticky reprezentovaných tvarů (např. v ray-tracingu), v realtime grafice se 2D texturové souřadnice uvádějí explicitně v attributech vrcholů modelu.

3D textura reprezentuje přímo vnitřní strukturu materiálu, textura je zobrazením/předpisem s 3D definičním oborem.

Příklady 2D textur: tapeta, nálepka, vyfotografovaný povrch reálného předmětu.

Příklady 3D textur: dřevo, mramor, porézní materiály.

2. Výhody 3D textury: nemusí se mapovat, 3D souřadnice průsečíku (ray-based rendering) nebo fragmentu (GPU) se přímo použijí jako argumenty textury.

Nevýhody: při reprezentaci tabulkou (rastrem) je velká spotřeba paměti (a tím nepřímo i menší rychlost). Při použití šumových funkcí se musí počítat ve 3D, což může zpomalit výpočet. Menší nevýhoda: při animaci si musíme dát pozor na souřadnou soustavu, ve které texturu používáme, ta se musí pohybovat spolu s animovaným objektem, v případě deformací objektu musíme implementovat příslušnou deformaci definičního oboru textury (to nemusí být vůbec jednoduché).

3. Příklad simulace dřeva nebo mramoru:

Namodelujeme si nejdříve vnitřní strukturu daného materiálu v ideálním případě – u dřeva by to byly ideálně soustředné válcové plochy letokruhů, u mramoru dokonale rovnoběžné vrstvy jednotlivých minerálů. V obou případech to znamená mít prostorově definovanou funkci $C_{ideal} : [x, y, z] \mapsto [R, G, B]$.

Pak použijeme nějakou prostor deformující funkci (funkce), kterými vstupní prostor nahodile křivíme tak, aby to co nejlépe odpovídalo struktuře přírodního materiálu. Používají se generátory spojitého šumu (např. slavný Perlinův šum) nebo se syntetizuje umělá turbulence pomocí několika složek běžného šumu (zvyšující se frekvence a snižující se amplituda).

Nechť máme tři různé deformující (šumové) funkce N_x , N_y a N_z , kde např.

$$N_x : [x, y, z] \mapsto x'$$

pak výslednou šumovou texturu postavíme jako

$$C_{wood}(x, y, z) = C_{ideal}(N_x(x, y, z), N_y(x, y, z), N_z(x, y, z))$$

19 GPU - data a shadery (specializace PGVVH-VPH)

Budeme se zabývat daty, které aplikace posílá do GPU a tzv. shadery (shaders), které programátoři moderních GPU musí připravit.

1. Napište, které typy shaderů se v běžném zobrazování 3D grafiky používají (neuvádějte shadery pro RT a GPGPU). U každého typu uveďte, v jaké části zobrazovacího řetězce je zařazen, a jaké hlavní funkce plní. Jsou některé typy shaderů povinné a nesmějí chybět v žádné aplikaci?
2. Jakou formou se předávají data z aplikace do GPU? Uvažujte opravdu všechny typy dat, vstupy shaderů, geometrická data scény, data popisující vzhled (materiály)... Jakým způsobem se všechna tato data do GPU předávají? (Když pomineme shadery, které by se také daly považovat za data, měli byste uvést minimálně tři formy předávání dat). U každé formy odlište, k jakému použití se hodí nejvíce, například, jak často se v aplikaci mění (nastaví se jednou provždy, mění se jednou za mnoho sekund, mění se každý snímek/frame, mění se mnohokrát za snímek/frame).
3. Kdybyste měli v zobrazovacím řetězci na GPU nějaký časově náročný výpočet (například pseudo-simulaci vodní hladiny nebo nějakou složitou procedurální texturu), jak byste se ho snažili optimalizovat? Do které části zobrazovací pipeline byste se ho snažili umístit? Doplňující/návodná otázka: které shadery na GPU spotřebovávají největší část výpočetního výkonu?

Nástin řešení 1. Shadery, jak jsou za sebou v zobrazovacím řetězci:

Vertex shader - zpracovává všechna data přiřazená k vrcholu. Je povinný. Musí spočítat minimálně souřadnice vrcholu v “clip space”. Typicky provádí Modelovou (model), Pohledovou (view) a Projekční (projection) transformaci a všechna ostatní streamová data vrcholu předává dál, aby je mohly později použít ostatní shadery.

Hull shader - nepovinný, přijímá indexy a připravuje data pro teslační stupně.

Tessellation shaders - nepovinné dva stupně pro teselaci kreslené geometrie, pro celkem pravidelné topologie sítě. “Tessellation Control Shader” konfiguruje celkovou topologii sítě a “Tessellation Evaluation Shader” počítá polohy vrcholů.

Geometry shader - nepovinný, může ještě mnohem obecněji modifikovat kreslená primitiva, může vytvářet zcela nové objekty (třeba z každého vrcholu může vyrobit malou trojúhelníkovou síť - např. dvacetistěn). Má přístup k datům celého vstupního grafického primitiva, na výstup posílá také celá grafická primitiva.

Fragment shader - určuje barvu výsledného fragmentu (potenciálního pixelu), je povinný. Typicky k tomu využívá vhodné vstupní streamové veličiny (viz Vertex shader), globální konstanty (uniforms), textury, apod.

2. Tzv. Buffers - speciálně Vertex buffers (VB), obsahují streamové veličiny popisující jednotlivé vrcholy geometrie. Vertex shader dostane vždy jen data jednoho vrcholu.

Index buffers (IB) - obsahují indexy popisující grafická primitiva (ze kterých vrcholů se skládají). K těmto datům má přístup stupeň “primitive assembly”, případně Hull shader, pokud je použit.

Textury - speciální buffery obsahující rastrová data, ke kterým mají přístup shadery. Textura je vždy přístupná jako celek, je k dispozici několik pokročilých interpolačních technik včetně MIP-mappingu (tj. HW se nám stará o sampling a interpolaci).

Uniforms/konstanty/Constant buffers - globální proměnné pro shadery. Pro jednu zobrazovací dávku (render call) jsou tyto hodnoty konstantní a mají k nim přístup všechna GPU jádra realizující daný výpočet. Typicky tam mohou být odkazy na textury, souřadnice světelných zdrojů, transformační matice pro Vertex shader, materiálové konstanty pro Fragment shader, apod.

Jak často se mění:

VB a IB - je snahou, aby se nahrály do paměti GPU a pak už se nemusely měnit (třeba několik minut, podle charakteru aplikace).

Textury - pokud možno by se měly také vejít do paměti GPU, ideálně se mění jen při nějaké velké změně vykreslované scény (nový level hry, vrtulník vs. chůze po zemi, vstup do podzemního komplexu...).

Uniforms - některé se mění jen jednou za čas, některé přesně jednou za snímek/frame, některé souvisí s kreslenou dávkou nebo instancí modelu a musí se měnit mnohokrát za snímek.

3. Náročný výpočet je dobré buď provést jenom jednou (např. Compute shaderem) a výsledek si uložit do sdíleného bufferu/textury, odkud se pak bude jen číst.

Pokud to není možné, tak je dobré co nejvíce náročného počítání přenést do Vertex shaderu, protože ten se nevyvolává tak často. Nevýhodou je však, že se spočítaná veličina už pak jenom umí lineárně interpolovat do fragmentů.

Pokud ani to není možné, musí se výpočet nechat ve Fragment shaderu. Bude se spouštět pro každý pixel. To je taky odpověď na otázku, co nejvíc výpočetně zatěžuje GPU - Fragment shadery.

20 Spouštění generovaného strojového kódu (specializace PVS)

Předpokládejte, že programujeme základ JIT překladače pro architekturu x64 (x86-64/AMD64) napsaného v C++, který má generovat strojový kód v kontextu nějakého managed prostředí jako .NET nebo Java. Abychom si ověřili schopnost generovat a spustit generovaný strojový kód, napsali jsme si následující krátký program:

```
int main()
{
    uint8_t machineCode[] =
    {
        0x8B, 0xC1, // MOV EAX, ECX
        0x03, 0xC2, // ADD EAX, EDI
        0xC3      // RET
    };

    auto func = reinterpret_cast<int32_t(*)>(int32_t, int32_t>(&machineCode[0]));

    int32_t result = func(30, 12);
    cout << "result: " << result << "\n";

    return 0;
}
```

Pokud tento program přeložíme pomocí C++ překladače X a spustíme na Windows, tak program **nevypíše** očekávanou hodnotu 42, ale spadne s chybou: 0xc0000005 (Access Violation)

Pokud ho přeložíme pomocí C++ překladače Y a spustíme na Linuxu, tak program také **nevypíše** očekávanou hodnotu 42, ale spadne s chybou: Segmentation fault (core dumped)

1. Vysvětlíte, proč daná chyba vzniká, a proč program nefunguje dle očekávání.
2. Ve Windows existuje funkce `VirtualProtect`, a na Linuxu podobná funkce `mprotect`, jejichž vhodné zavolání náš problém vyřeší. Načrtněte, jaké parametry musí taková funkce minimálně mít. Pro každý z parametrů uveďte, jaký je jeho význam a typ.
3. Napište na jakém místě a přesně jak (s jakými parametry) byste uvedenou funkci zavolali tak, aby výše uvedený program začal na Windows fungovat a vypsal hodnotu 42. Pomiňme nebezpečnost takového postupu na uvedeném jednoduchém programu.

4. Pokud je program s úpravou z části 3 „přenesen“ na Linux (tj. volání funkce `VirtualProtect` nahradíme ekvivalentním voláním `mprotect`), program na Linuxu přeložíme a spustíme, tak nespadne, ale místo očekávané hodnoty 42 vypíše hodnotu -986578528. Vysvětlíte, proč se program takto chová, a co způsobilo tuto závadu. Co by se muselo v programu upravit, aby fungoval na Linuxu (stačí načrtnout princip úpravy, není třeba ji popisovat zcela přesně)?

Nástin řešení

1. Obsah pole `machineCode` leží na zásobníku nebo ve statických datech, tyto stránky mají ve Windows i Linuxu nastavený příznak NX (No eXecute).
2. Adresa začátku stránky (tj. dělitelná beze zbytku velikostí stránky) nebo adresa ve stránce (dle nuancí API operačního systému) = pointer; typicky i délka bloku paměti, u jehož stránek se mají nastavit příznaky ochrany stránek; maska příznaků ochrany stránky (typicky minimálně READ, WRITE, EXECUTE).
3. Stačí kdekoliv před voláním `func(30, 12)`. Např. pro reálný `VirtualProtect` (v řešení má být: adresa `machineCode`, resp. stránky s `machineCode`; velikost bufferu `machineCode`; maska zahrnující příznak EXECUTE):

```
DWORD oldProtect;
bool success = VirtualProtect(
    machineCode,
    sizeof(machineCode),
    PAGE_EXECUTE_READWRITE,
    &oldProtect
);
```

4. Překladač Y na Linuxu zřejmě používá jinou volací konvenci než překladač X na Windows - tj. parametry 30 a 12 jsou předané funkci v jiných registrech než ECX a EDI, které očekává „vygenerovaný“ strojový kód. Mohli bychom např. opravit strojový kód, aby používal správné registry dle volací konvence na Linuxu.

21 r-value reference (specializace PVS)

Stručně (cca 4-5 řádků), vysvětlíte princip, fungování a typické použití r-value referencí a move sémantiky v C++.

V následujícím fragmentu kódu třída `Buffer` vlastní dynamicky alokované pole znaků.

```
class Buffer {
    char* data_;
    std::size_t size_;

public:
    Buffer() : data_(nullptr), size_(0) {}
    explicit Buffer(std::size_t size) : data_(new char[size]), size_(size) {}
    ~Buffer() { delete[] data_; }

    Buffer(const Buffer&) = delete;
    Buffer& operator=(const Buffer&) = delete;

    // move constructor
    // move assignment operator

    char* data() { return data_; }
    std::size_t size() const { return size_; }
};
```

Doplňte implementaci:

- move konstruktoru
- move přiřazovacího operátoru

Po přesunu má být zdrojový objekt v bezpečném prázdném stavu stejném jako po defaultním konstruktoru.

Dále stručně vysvětlete, proč jsou defaultní move metody nevhodné a proč je zakázán copy konstruktor a copy přiřazovací operátor.

```
Buffer::Buffer(Buffer&& other) noexcept
    : data_(other.data_),
      size_(other.size_)
{
    other.data_ = nullptr;
    other.size_ = 0;
}

Buffer& Buffer::operator=(Buffer&& other) noexcept
{
    if (this != &other) {
        delete[] data_;

        data_ = other.data_;
        size_ = other.size_;

        other.data_ = nullptr;
        other.size_ = 0;
    }

    return *this;
}
```

Kopírování je zakázané proto, že Buffer vlastní ukazatel na data. Automaticky vygenerované kopírování by pouze zkopírovalo hodnotu ukazatele, takže dva objekty by vlastnily stejnou paměť. Při destrukci by pak došlo k dvojímu delete[].

Move konstruktor a move přiřazení naopak vlastnictví paměti pouze přesunou z jednoho objektu do druhého. Zdrojový objekt se nastaví na nullptr, takže jeho destruktor už nic nesmaže.

22 Databáze knihovny (specializace PVS)

Předpokládejme následující relační schéma **databáze knihovny**:

Ctenar (rodneCislo, jmeno, prijmeni, vek, mesto)

Primární klíč: rodneCislo

Kniha (signatura, nazev, autor, zanr)

Primární klíč: signatura

Vypujcky (id, datumPujceni, kniha, rodneCislo, datumVraceni)

Primární klíč: id

Unikátní klíč: datumPujceni, kniha, rodneCislo

Cizí klíč: Vypujcky[knaha] $\subseteq$ Kniha[signatura]

Cizí klíč: Vypujcky[rodneCislo] $\subseteq$ Ctenar[rodneCislo]

Napište SQL SELECT výrazy pro následující dotazy:

- Najděte unikátní příjmení čtenářů starších 30 let, kteří si nikdy nepůjčili žádnou knihu patřící do žánru *drama* nebo *kuchařka*. Použijte predikát IN.
- Najděte rodná čísla, jména a příjmení čtenářů, kteří alespoň jednou měli ve stejný okamžik vypůjčeny alespoň dvě knihy najednou. Použijte predikát EXISTS.
- Pro každou knihu od autora *Franz Kafka* najděte celkový počet výpůjček realizovaných čtenáři z města *Liberec*.

Nástin řešení

```
SELECT DISTINCT C.prijmeni
FROM Ctenar AS C
WHERE
  (C.vek > 30) AND
  (C.rodneCislo NOT IN (
    SELECT V.rodneCislo
    FROM Vypujcka AS V JOIN Kniha AS K ON (V.kniha = K.signatura)
    WHERE (K.zanr IN ('drama', 'kuchařka'))
  ));

SELECT C.rodneCislo, C.jmeno, C.prijmeni
FROM Ctenar AS C
WHERE EXISTS (
  SELECT *
  FROM Vypujcka AS V1 JOIN Vypujcka AS V2 ON (V1.rodneCislo = V2.rodneCislo) AND (V1.id < V2.id)
  WHERE
    (V1.rodneCislo = C.rodneCislo) AND
    (V1.datumPujceni <= V2.datumVraceni) AND (V1.datumVraceni >= V2.datumPujceni)
);

SELECT K.signatura, COUNT(V.id) as pocetVypujcek
FROM
  Kniha AS K LEFT OUTER JOIN (
    Vypujcka AS V NATURAL JOIN Ctenar AS C
  ) ON (K.signatura = V.kniha) AND (C.mesto = 'Liberec')
WHERE (K.autor = 'Franz Kafka')
GROUP BY K.signatura;
```

23 (Ne)bezpečné fórum (specializace PVS)

Mějme jednoduchou webovou aplikaci typu diskusní fórum. Uživatel může na veřejné stránce odeslat komentář, který se uloží do databáze. Stejná stránka následně vypíše všechny uložené komentáře.

Aplikace je implementována takto:

```
<?php
$db = new mysqli("localhost", "forum_user", "password", "forum");

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $author = $_POST["author"];
    $text = $_POST["text"];
    $sql = "INSERT INTO comments (author, text) VALUES ('$author', '$text')";
    $db->query($sql);
}

$result = $db->query("SELECT author, text FROM comments ORDER BY priority DESC, id DESC");
?>

<form method="post">
    Jméno: <input name="author">
    Komentář: <textarea name="text"></textarea>
    <button type="submit">Odeslat</button>
</form>

<h2>Komentáře</h2>

<?php while ($row = $result->fetch_assoc()) { ?>
```

```

<div class="comment">
    <b><?php echo $row["author"]; ?></b><br>
    <?php echo $row["text"]; ?>
</div>
<?php } ?>

```

Databázová tabulka může mít například tuto strukturu:

```

CREATE TABLE comments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    author VARCHAR(100),
    text TEXT,
    priority INT DEFAULT 0
);

```

Sloupec `priority` není dostupný ve formuláři a běžně jej nastavuje pouze administrátor.

1. Posuďte, zda je aplikace zranitelná vůči útokům založeným na vstupu od uživatele. Identifikujte relevantní zranitelnosti a vysvětlete jejich princip.
2. Pro každou nalezenou zranitelnost popište možný scénář zneužití (např. jak může vypadat uživatelský vstup a jak se následně útok projeví). Pokud možnost realizace závisí na dalších okolnostech (např. konfiguraci serveru nebo konkrétní vlastnosti použitých funkcí), vysvětlete proč.
3. Navrhněte úpravy aplikace, které nalezené zranitelnosti odstraní.

Nástin řešení V odpovědi se očekává především zmínka SQL injection a script injection.

1. SQL injection je útok, při kterém útočník vloží speciálně upravený vstup, který se stane součástí SQL dotazu a změní jeho význam. Aplikace je vůči SQL injection potenciálně zranitelná, protože vstupy `$author` a `$text` jsou bez jakéhokoli ošetření vloženy přímo do SQL řetězce. Konkrétní dopad závisí na konfiguraci databáze, především na tom, jestli `$db->query($sql)` umožní zpracovat vícero dotazů najednou. Pokud ano, tak např. `Petr')`; `DROP TABLE comments;` -- by vedl ke smazání tabulky.
2. Script injection (XSS) spočívá v tom, že útočník vloží do aplikace HTML nebo JavaScript kód, který se následně vykoná v prohlížeči jiného uživatele. Aplikace je vůči tomuto útoku zranitelná, protože komentáře jsou při výpisu vkládány přímo do HTML stránky: `<?php echo $row["text"]; ?>`. Útočník může například vložit komentář: `<script>alert('XSS')</script>`, který se při následném načtení stránky vykoná. V reálném útoku by mohl například provádět akce jménem přihlášeného uživatele nebo se pokoušet získat citlivé informace.
3. Proti SQL injection je vhodné použít parametrizované dotazy (prepared statements), například pomocí `$db->prepare()`. Alternativně je možné využít odpovídající escapovací funkce, např. `$author = mysqli_real_escape_string($db, $_POST["author"]);` Proti XSS je nutné při výpisu dat do HTML provést escapování speciálních znaků, například pomocí funkce `htmlspecialchars()`. Další vhodná opatření zahrnují validaci vstupů, omezení databázových oprávnění, použití Content Security Policy (CSP) a nastavení autentizačních cookies s atributem `HttpOnly`.

24 Lexikální analýza (specializace SP)

1. Stručně popište účel lexeru v rámci překladač.
2. Navrhněte datové typy pro lexer, který má rozpoznat aritmetické výrazy s `+` a `*` nad kladnými celými čísly (tj. bez závorek či proměnných).
3. Pseudokódem načrtněte implementaci lexeru bez využití regulárních výrazů či jiných funkcí, které zpracovávají více než jeden znak. Další znak na vstupu získejte voláním `next_char`, pro klasifikaci znaku využijte funkce jako jsou `is_digit` nebo `is_blank`.

Jaký bude návratový typ hlavní funkce, kterou bude volat parser? Je přípustná jak varianta, která přečte celý vstup najednou, tak i varianta, která čte vstup podle potřeby.

Chování funkce `next_char` pro konec vstupu si definujte rozumným způsobem (dané chování popište a zdůvodněte).

Nástin řešení Lexer bude pracovat s tokeny, které budou PLUS, TIMES, NUMBER (s přidruženým atributem zpracovaného čísla) a EOF.

```
Token next() {
    while (true) {
        c = next_char();
        if (is_digit(c)) {
            num = char_digit_to_int(c);
            while (is_digit(c := next_char ())) {
                num = num * 10 + char_digit_to_int(c);
            }
            yield Token.makeNumber(num);
            if (c == EOT) break;
            // already read next char, cascade
        }
        if (c == EOT) break;
        if (is_blank(c)) continue;
        if (c == '+') {
            yield Token.make(PLUS);
        } else if (c == '*') {
            yield Token.make(TIMES);
        } else {
            throw BadInput("Unexpected character " + c)
        }
    }
    return Token.make EOF;
}
```

25 Podmínková proměnná (specializace SP)

1. Popište sémantiku následujících tří volání nad podmínkovou proměnnou (CV, condition variable).

```
cv_init(cv_t *cv);
cv_signal(cv_t *cv);
cv_wait(cv_t *cv, mutex_t *mtx);
```

2. Předpokládejme, že implementace je určena pro jádro operačního systému na počítačích s jedním procesorem.

K dispozici máte funkce pro uspaní aktuálního vlákna (`thread_suspend`), probuzení daného vlákna (`thread_wakeup`) a pro získání aktuálního vlákna (`thread_get_current`). Kromě toho jsou pochopitelně k dispozici operace pro uzamčení a odemčení mutexu a běžné datové struktury jako kolekce (které ale nemají zaručenou funkci při současném volání více vláken).

Napište pasivní implementaci condition variable založenou na výše uvedených funkcích.

3. Jak by bylo nutné implementaci upravit, aby fungovala v uživatelském prostoru (neuvažujeme alternativu, kdy půjde o specializovanou systémovou volání přímo pro podmínkovou proměnnou)?

Není nutné psát kód, ale popište, jaké funkce (včetně signatury) byste od jádra operačního systému vyžadovali (zahrňte případně i funkce `thread_suspend` a `thread_wakeup`, pokud je vaše implementace potřebuje).

Nástin řešení `cv_signal` probudí jedno vlákno čekající v `cv_wait`. `cv_wait` uspí aktuální vlákno do volání `cv_signal` a zároveň atomicky odemkne mutex. Po návratu z této funkce je mutex ale opět uzamčený.

Atomicitu na single-CPU stroji zajistí (v kernelu) např. jen vypnutí interruptů.

```
cv_t:
    list_t waiters
```

```

init:
    cv->waiters = empty_list

wait:
    atomically:
        list_add(cv->waiters, thread_get_current)
        mutex_unlock(mtx)
        thread_suspend()
    mutex_lock(mtx)

signal:
    atomically:
        if not list_empty(cv->waiters):
            first = list_pop(cv->waiters)
            thread_wakeup(first)

```

26 Síťový provoz (specializace SP)

Spuštění tcpdump na Linuxovém serveru s IP adresou 195.113.20.170 vypsalo následující:

```

1.2.3.4:42384 > 195.113.20.170:80 [S ]
195.113.20.170:80 > 1.2.3.4:42384 [SA ]
1.2.3.4:42384 > 195.113.20.170:80 [PA ] GET /serverinfo HTTP/1.0
195.113.20.170:80 > 1.2.3.4:42384 [PA ] HTTP/1.1 404 Not found
1.2.3.4:42384 > 195.113.20.170:80 [FA ]
195.113.20.170:80 > 1.2.3.4:42384 [FA ]
5.6.7.8:47457 > 195.113.20.170:80 [S ]
195.113.20.170:80 > 5.6.7.8:47457 [SA ]
5.6.7.8:47457 > 195.113.20.170:80 [PA ] GET /about.html HTTP/1.0
195.113.20.170:80 > 5.6.7.8:47457 [PA ] HTTP/1.1 200 OK
195.113.20.170:80 > 5.6.7.8:47457 [PA ] \r\n ...
1.2.3.4:45628 > 195.113.20.170:80 [S ]
195.113.20.170:80 > 1.2.3.4:45628 [SA ]
1.2.3.4:45628 > 195.113.20.170:80 [PA ] POST /login HTTP/1.0
195.113.20.170:80 > 1.2.3.4:45628 [PA ] HTTP/1.1 404 Not found
195.113.20.170:80 > 5.6.7.8:47457 [PA ] <html> ...
5.6.7.8:47457 > 195.113.20.170:80 [A ]
5.6.7.8:47457 > 195.113.20.170:80 [FA ]
1.2.3.4:45628 > 195.113.20.170:80 [FA ]
195.113.20.170:80 > 1.2.3.4:45628 [FA ]
195.113.20.170:80 > 5.6.7.8:47457 [FA ]
1.2.3.4:42817 > 195.113.20.170:80 [S ]
195.113.20.170:80 > 1.2.3.4:42817 [SA ]
1.2.3.4:42817 > 195.113.20.170:80 [PA ] POST /admin HTTP/1.0
195.113.20.170:80 > 1.2.3.4:42817 [PA ] HTTP/1.1 404 Not found
1.2.3.4:42817 > 195.113.20.170:80 [FA ]
195.113.20.170:80 > 1.2.3.4:42817 [FA ]
1.2.3.4:48342 > 195.113.20.170:80 [S ]
1.2.3.4:44521 > 195.113.20.170:80 [S ]

```

V dumpu identifikujte stroje, které se komunikace účastnily a rozepište, na kterých portech došlo ke spojení a co bylo pravděpodobně předmětem komunikace.

Pokuste se přibližně odvodit význam flagů S, A, P a F.

Co můžeme usoudit z posledních dvou paketů (řádků)?

Vysvětlíte, co se mohlo na serveru stát a proč, pokud jsme ihned po předchozí komunikaci zachytili následující (tj. porovnejte, co můžeme usoudit na stav serveru na základě pouze první části dumpu, a co pokud máme k dispozici i druhou část).

```
5.6.7.8:41952 > 195.113.20.170:80 [S ]
195.113.20.170:80 > 5.6.7.8:41952 [SA ]
5.6.7.8:41952 > 195.113.20.170:80 [PA ] GET /favicon.ico HTTP/1.0
195.113.20.170:80 > 5.6.7.8:41952 [PA ] HTTP/1.1 200 OK
5.6.7.8:41952 > 195.113.20.170:80 [FA ]
195.113.20.170:80 > 5.6.7.8:41952 [FA ]
```

Nástin řešení Na stroji 195.113.20.170 běží webový server na portu 80. Stroj 1.2.3.4 postupně zkouší stahovat „podezřelé“ stránky (jednotlivá spojení jdou na jiných portech, čili se nevyužívá žádná pokročilejší vlastnost protokolu HTTP). Stroj 5.6.7.8 si stahuje stránku `about.html`.

Flagy odpovídají typům paketů v TCP komunikaci, čili počáteční handshake, přenos dat (push) a ukončení komunikace. Flag A pak označuje potvrzení aktuální pozice v přenosu, které je po prvním SYN součástí paketu.

Poslední dva řádky obsahují jen zahájení konverzace ze strany klienta. Protože `tcpdump` běží na daném stroji, pravděpodobně došlo k vypnutí/pádu webovou serveru.

Protože ale následující komunikace s klientem 5.6.7.8 opět probíhá, byl buď server restartován (a některé pakety se ztratily) nebo, což je vzhledem k povaze dotazů pravděpodobnější, byly pakety z adresy 1.2.3.4 nějakým způsobem zablokovány (např. službou typu `fail2ban`).

27 Stránkování (specializace SP)

Uvažujme 16-bitový RISC procesor s jednoúrovňovou tabulkou pro překlad virtuálních adres.

Uvažujme následující zjednodušení:

- Virtuální i fyzické adresy mají velikost 16 bitů.
- Fyzická paměť má maximálně 32KB a je k dispozici vždy od adresy 0 (tj. polovina maximální adresovatelné velikosti).
- Kernel pro svoje potřeby (kód, data) využívá vždy jen virtuální adresy ve spodních 32KB.
- Adresy v kernelu jsou mapovány vždy 1:1 (to nebrání je využít i pro uživatelská data!).
- Uživatelský prostor (aplikace) jsou vždy spuštěny v „horních“ 32KB.

Překladová tabulka je volně inspirována RISC-V Sv32, je procházena hardwarem a každá položka má 16bitů s následující strukturou (nejvyšší bit není využit, implementace ho ale musí nechat nastavený na 0).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	ASID					PPN					User	Exec	Write	Read	Valid

Adresové prostory pro aplikace v uživatelském režimu mohou mít několik oblastí, které jsou definovány strukturou `as_area_t`, kde položky `offset` a `size` jsou indexy (tj. pracují s jednotkou jedna stránka).

```
typedef struct {
    // Adresa strankovaci tabulky tohoto adresoveho prostoru
    uint16_t *table;
    // Dalsi detaily nejsou pro tuto otazku potreba
    ...
} as_t;
```

```
typedef struct {
    uint16_t offset;
    uint16_t size;
} as_area_t;
```

V kontextu této architektury (kernelu) vyřešte následující.

1. Jak velké jsou jednotlivé stránky a jakou velikost bude mít stránkovací tabulka nulté úrovně?

2. Pomocí pseudokódu implementujte funkci `on_user_page_fault`, která je volána z obsluhy přerušení při výpadku stránky. K dispozici máte funkci `as_area_t *as_area_find(as_t*, uint16_t virt)`, která vrací oblast odpovídající dané adrese, případně NULL, pokud taková oblast neexistuje, a funkci `uint8_t as_get_asid(as_t*)`, která vrací ASID pro daný adresový prostor a vždy uspěje.

Pokud je adresa neplatná nebo není k dispozici žádná volná paměť, proces ukončete voláním `kill_current_process`, v opačném případě aktualizujte překladovou tabulku, která je k dispozici jako `as->table` (vizte omezení výše pro přístup k tabulce). Nový rámec alokujte pomocí volání `frame_alloc`, které vrací fyzickou adresu nebo -1 při nedostatku paměti.

Novou položku do tabulky přidejte vždy s oprávněními pro čtení a zápis, stránka leží v uživatelské prostoru (kernelové stránky jsou vždy namapovány identicky a nemusíte je při výpadku stránky řešit).

```
void on_user_page_fault(as_t *current_as, uint16_t fault_address) {
    ...
}
```

Nástin řešení Pro 16b fyzické adresy a 6b PPN zbývá 10b na offset, to odpovídá 1kB stránkám. Ještě je možné vzít jako výchozí informaci max 32kB fyzické paměti, pak stejná úvaha dojde k 512B stránce.

S 1kB stránkami a 2B položkami stránkovacími tabulky je na stránkovací tabulku potřeba 128B, s 512B stránkami by to bylo 256B. Stejnou logikou ale jde zdůvodnit také polovina, pokud uvažíme, že se překládají pouze uživatelské adresy v horních 32kB.

```
on_user_page_fault(current_as, fault_address):
    // atomicky
    area = as_area_find(current_as, fault_address)
    if area == NULL:
        kill_current_process()
        return
    new_frame = frame_alloc()
    if new_frame == -1:
        // OOM
        kill_current_process()
        return
    ppn = new_frame >> 10 // page size
    access = 0b10111 // valid, r, w, user
    asid = as_get_asid(current_as)
    entry = access | (ppn << 5) | (asid << 11)
    vpn = fault_address >> 10 // page size
    current_as->table[vpn] = entry
```

28 Pravděpodobnostní minové pole (specializace UI-SU, UI-ZPJ, UI-ROB)

Popis. Uvažte variantu hry Minesweeper na obdélníkové mřížce $m \times n$. Na rozdíl od klasické verze není celkový počet min předem dán. Každé pole nezávisle obsahuje minu s apriorní pravděpodobností $p \in (0, 1)$.

V průběhu hry jsou odhalena některá pole. Odhalené pole ukazuje přesný počet min mezi svými osmi sousedy (vodorovně, svisle a diagonálně). Odhalená pole jsou zaručeně bez min.

Pro daný stav hry, který sestává z odhalených čísel a neprozkoumaných polí, je vaším úkolem určit pro každé neprozkoumané pole *a posteriori pravděpodobnost*, že obsahuje minu, podmíněnou všemi pozorováními.

Otázky.

1. Napište SAT (formule v konjunktivně normální formě) nebo CSP (splnění omezujících podmínek) model, který určí, zda dané pole zcela jistě neobsahuje minu. Model vysvětlete.
2. Nechť U je množina neprozkoumaných polí a $\mathbf{x} \in \{0, 1\}^{|U|}$ označuje konkrétní konfiguraci min, kde $x_c = 1$ znamená, že pole $c \in U$ obsahuje minu. Nechť O je pozorovaný stav hry a $\mathcal{C} \subseteq \{0, 1\}^{|U|}$ je množina všech konfigurací konzistentních s odhalenými čísly.

Napište obecný vzorec pro podmíněnou pravděpodobnost, že dané neprozkoumané pole $c \in U$ obsahuje minu:

$$P(X_c = 1 \mid O).$$

3. V následujících příkladech vypočítejte pravděpodobnost miny na každém poli. Výsledky můžete napsat přímo do tabulek.

a)

1		

Pokračuje na další stránce.

Student 666. Pokračování otázky *Pravděpodobnostní minové pole*.

b)

1		1

c)

	2	1
2		
1		

29 Vylepšení spamového filtru (specializace UI-SU, UI-ZPJ)

Převzali jste vývoj spam filtru ve vaší organizaci. Uživatelé si stěžují, že funguje špatně. Po bližším prozkoumání jste zjistili, že váš předchůdce natrénovat klasifikátor logistické regrese s příznaky TF-IDF na datasetu, kde 2% e-mailů bylo spam a zbytek běžná pošta. Na těchto trénovacích datech model dosahuje 98% úspěšnosti (accuracy).

Uživatelé ve vaší organizaci pečlivě označují veškerý spam, který obdrží. Když se podíváte na některé označené spamy, vidíte

příklady, které vypadají nějak takto:

```
Fr33 V1agr@ - kl!kn1 zd3 pr0 Objedn@vku!!!  
Vyhr@11 jst3 p0uk@zku v h0dn0t3 1000 Kč! Zn@nte n0w: www.fr33-cen@.cz  
URGENTNÍ: V@š @cc0unt byl p0z@st@ven. Ověř1t3 okamž1tě na http://s3cur3-pr1hl@sen1.net  
Vydel@jte r0zdel@n3 pen1ze!!! Pr@cujte z dom0va €€€ - L3G1T N@B1DK@!!!
```

Po prozkoumání TF-IDF reprezentace těchto zpráv jste zjistili, že většina slov není ve vašem slovníku, takže se vůbec neobjeví ve vektoru příznaků TF-IDF, nebo jsou tokenizována na jednotlivé znaky, které samy o sobě vypadají neškodně.

Z důvodu efektivity nemáte možnost nahradit pipeline logistické regrese s TF-IDF, ale máte výpočetní kapacitu na implementaci několika jednoduchých dodatečných příznaků (features).

1. Váš předchůdce reportuje 98 % úspěšnost (accuracy) na trénovacích datech a usuzuje, že model je výborný. Vysvětlete, proč je tento závěr chybný. Jaké je nejpravděpodobnější chování klasifikátoru a jaká je nejpravděpodobnější příčina?
2. Navrhněte **tři konkrétní typy dodatečných příznaků** (features), které byste mohli přidat vedle TF-IDF reprezentace, aby klasifikátor lépe detekoval tento typ spamu. U každého typu příznaků vysvětlete, co zachycuje a proč je užitečný.
3. Po přetrénování modelu s novými příznaky chcete pořádně vyhodnotit, zda je nový model lepší než starý. Z tisíců emailů, které přišly, uživatelé označili 400 e-mailů jako spam. Popište, jak byste toto vyhodnocení provedli. Jaké metriky byste použili a proč? Byla by úspěšnost (accuracy) dobrá volba?

Nástin řešení

1. Závěr je chybný ze dvou důvodů. Zaprvé, trénovací data jsou silně nevyvážená: 2 % spamu znamená, že klasifikátor, který označí *každý* e-mail jako legitimní, dosáhne 98 % přesnosti triviálně, aniž by se cokoli naučil. Zadruhé, i kdyby se model něco naučil, přesnost na trénovacích datech není platným odhadem generalizačního výkonu. Vždy je nutné vyhodnocovat na oddělených datech. Nejpravděpodobnější chování modelu je, že klasifikuje vše jako legitimní poštu (nebo téměř vše), protože to minimalizuje trénovací chybu na nevyvážených datech.
2. Tři rozumné příklady (jiné dobře zdůvodněné odpovědi jsou také přijatelné):
 - **Frekvence méně obvyklých znaků:** podíl znaků, které jsou číslice nebo speciální znaky používané místo písmen (např. 0, @, 1, 3, !). Autoři spamu je používají, aby obešli filtry klíčových slov, ale zpráva zůstane čitelná pro člověka.
 - **Míra slov mimo slovník (OOV):** podíl tokenů, které se nenacházejí ve slovníku TF-IDF. Legitimní e-maily zřídka obsahují mnoho neznámých tokenů, zatímco obfuskovaný spam jich obsahuje mnoho. Takový rys může být problematický, pokud někteří uživatelé komunikují ve více jazycích.
 - **Přítomnost URL / podezřelé URL:** binární příznak přítomnosti URL, nebo podrobnější příznaky jako to, zda doména obsahuje číslice či pomlčky, nebo využívá bezplatný hosting. Spamové e-maily velmi často obsahují URL odkazující na podvodné stránky.

Další rozumné příznaky: podíl velkých písmen, počet vykřičníků, přítomnost symbolů měn, poměr číslic k písmenům.

3. Úspěšnost (accuracy) by byla špatnou volbou ze stejného důvodu jako výše: nevyváženost tříd znamená, že triviální klasifikátor dosahuje vysokého skóre. Lepšími metrikami jsou **přesnost (precision)**, **úplnost (recall)** a **F1-skóre** vypočítané pro třídu spam (pozitivní třída):

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}$$

30 Lineární regrese a regularizace (specializace UI-SU, UI-ZPJ)

Při trénování lineárního modelu s L2 regularizací jsme vyzkoušeli různé hodnoty λ a změřili trénovací a validační chybu (součet čtverců):

λ	Trénovací chyba	Validační chyba
0	1,2	4,5
0,5	1,4	3,0
1	1,8	2,5
2	2,5	2,7
5	4,0	3,5

1. Zapište optimalizační úlohy pro lineární regresi s L2 regularizací (Ridge) a s L1 regularizací (Lasso) s parametrem λ . Vysvětlete, jak λ ovlivňuje sílu regularizace v obou případech.
2. Kterou hodnotu λ byste vybrali na základě dat v tabulce výše? Zdůvodněte.
3. Co reprezentuje $\lambda = 0$? Proč má vysokou validační chybu, přestože trénovací chyba je nejnižší?
4. Co reprezentuje velké λ (např. $\lambda = 5$)? Proč opět roste validační chyba?
5. Jak byste implementovali hledání optimálního λ ?

Nástin řešení

1. Ridge regrese minimalizuje

$$\sum_{i=1}^n (\beta_0 + \sum_{j=1}^d \beta_j x_{ij} - y_i)^2 + \lambda \left(\sum_{j=1}^d \beta_j^2 \right),$$

tedy přidává L2 penalizaci $\lambda \|\beta\|_2^2$. Lasso minimalizuje

$$\sum_{i=1}^n (\beta_0 + \sum_{j=1}^d \beta_j x_{ij} - y_i)^2 + \lambda \left(\sum_{j=1}^d |\beta_j| \right),$$

tedy přidává L1 penalizaci $\lambda \|\beta\|_1$. Čím větší λ , tím silněji se penalizuje velikost koeficientů a tím jednodušší je výsledný model.

2. Nejlepší volba je $\lambda = 1$, neboť dosahuje nejnižší validační chyby (2,5).
3. $\lambda = 0$ odpovídá obyčejné OLS regresi bez regularizace. Model se může přeučit trénovacím datům, což se projeví vysokou chybou na validační množině.
4. Velké λ znamená silnou regularizaci. Koeficienty jsou příliš stlačeny k nule, model je příliš jednoduchý a podceňuje se. Validační chyba se tedy opět zhoršuje.
5. Můžeme použít k -násobnou křížovou validaci (crossvalidace) – data rozdělíme na k podmnožin (foldů). Pro každou hodnotu λ provedeme k běhů: v každém použijeme jeden fold jako validační množinu a zbývajících $k - 1$ foldů jako trénovací. Spočteme průměrnou validační chybu přes všechny foldy. Vybereme λ s nejnižší průměrnou validační chybou. Pokud máme k dispozici dostatek dat a model se trénuje dlouho lze použít i jen jednu validační množinu.

31 Náhodné lesy, bagging a porovnání modelů (specializace UI-SU)

1. Stručně popište myšlenku baggingu a algoritmus náhodného lesa (Random Forest). V čem se Random Forest liší od obyčejného baggingu nad rozhodovacími stromy?
2. Rozhodovací strom a náhodný les byly vyhodnoceny 5násobnou křížovou validací (5-fold CV). Na jednotlivých foldech byly naměřeny chybovosti (v procentech):

Fold	1	2	3	4	5
Strom	14	13	15	16	12
Náhodný les	10	12	11	13	9

Pomocí t -testu rozhodněte, zda je rozdíl mezi oběma modely statisticky významný na hladině 5 %. Kritická hodnota pro oboustranný test se čtyřmi stupni volnosti je $t_{\text{krit}} = 2,8$. Jakou variantu testu použijete a proč?

Nástin řešení

1. Bagging trénuje několik základních modelů (zde stromů) na různých bootstrap vzorcích z trénovacích dat a jejich predikce agreguje (hlasováním nebo průměrem). Random Forest kromě toho při každém dělení uzlu vybírá náhodnou podmnožinu příznaků.
2. Nejvhodnější je použití párového testu. Rozdíly $d_i = \text{Strom}_i - \text{Les}_i$: 4, 1, 4, 3, 3. Průměr:

$$\bar{d} = \frac{4 + 1 + 4 + 3 + 3}{5} = 3.$$

Výběrový rozptyl:

$$s^2 = \frac{(4-3)^2 + (1-3)^2 + (4-3)^2 + (3-3)^2 + (3-3)^2}{4} = \frac{1+4+1+0+0}{4} = \frac{3}{2}.$$

Směrodatná odchylka $s = \sqrt{3/2}$. Testová statistika:

$$t = \frac{\bar{d}}{s/\sqrt{n}} = \frac{3}{\sqrt{3/2}/\sqrt{5}} = \frac{3}{\sqrt{3/10}} = 3\sqrt{\frac{10}{3}} = \sqrt{30} \approx 5,48.$$

Absolutní hodnota $|t| \approx 5,48$ je větší než 2,8, proto na hladině 5 % zamítáme nulovou hypotézu o rovnosti středních chyb. Náhodný les je statisticky významně přesnější.

32 Evaluace detektoru citátů (specializace UI-ZPJ)

Představte si, že jste vytvořili systém pro automatické rozpoznávání slavných citátů v textu. Máte k dispozici referenční anotaci (zlatý standard), podle které chcete systém vyhodnotit.

Jako příklad si můžeme představit následující větu a referenční anotaci slavných citátů (označených jako rozsahy znaků):

Že pravda a láska zvítězí nad lží a nenávistí, chtěl určitě říct už Napoleon, ale neměl na to čas a navíc neuměl česky.

Referenční citát: pravda a láska zvítězí nad lží a nenávistí, rozsah znaků 3–44.

Systém 1 detekoval: pravda a láska zvítězí nad lží a nenávistí, rozsah znaků 3–44.

Systém 2 detekoval: Že pravda a láska zvítězí, rozsah znaků 0–24.

1. Jakou metriku byste použili pro vyhodnocení detektoru citátů, pokud se počítá celý citát nebo nic? Uveďte vzorec a vysvětlete jeho složky.
2. Jak byste přistoupili k vyhodnocení, pokud systém nedetekuje citáty celé, ale jen jejich části (jako v příkladu výše)? Navrhněte způsob, jak do evaluace zahrnout částečné shody, uveďte příslušné vzorce a ilustруйте je na příkladu výše.

Předpokládejte, že se citáty nepřekrývají.

Nástin řešení

1. Pro případ přesných shod (celý citát nebo nic) je přirozené použít **přesnost (precision)**, **úplnost (recall)** a **F1 skóre**, počítané přes všechny citáty v testovací sadě:

$$P = \frac{|\text{správně detekovaných citátů}|}{|\text{všech detekovaných citátů}|}, \quad R = \frac{|\text{správně detekovaných citátů}|}{|\text{všech referenčních citátů}|}$$

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

Samotná přesnost ani úplnost nestačí – systém může mít vysokou přesnost tím, že detekuje velmi málo citátů, což naopak sníží úplnost. F1 skóre obě hodnoty vyvažuje.

V příkladu výše: Systém 1 detekoval citát přesně, tedy $P = 1,0$, $R = 1,0$, $F_1 = 1,0$. Systém 2 detekoval jiný rozsah než referenční, tedy $P = 0,0$, $R = 0,0$, $F_1 = 0,0$.

2. Pro částečné shody je vhodné měřit překryv na úrovni znaků (nebo tokenů). Počet správně detekovaných znaků odpovídá délce průniku detekovaného a referenčního rozsahu. Přesnost a úplnost pak definujeme jako:

$$P_{\text{char}} = \frac{|\hat{s} \cap s^*|}{|\hat{s}|}, \quad R_{\text{char}} = \frac{|\hat{s} \cap s^*|}{|s^*|}$$

kde $\hat{s}$ je detekovaný rozsah, s^* je referenční rozsah a $|\cdot|$ označuje počet znaků. F1 skóre se počítá stejně jako výše. Při více citátech v testovací sadě se hodnoty agregují (např. makro- nebo mikro-průměrem).

Pro Systém 1: průnik rozsahů $[3, 44]$ a $[3, 44]$ má délku 42; $|\hat{s}| = 42$, $|s^*| = 42$. Tedy $P_{\text{char}} = 42/42 = 1,0$, $R_{\text{char}} = 42/42 = 1,0$, $F_{1,\text{char}} = 1,0$.

Pro Systém 2: průnik rozsahů $[0, 24]$ a $[3, 44]$ je $[3, 24]$, délka průniku je 22; $|\hat{s}| = 25$, $|s^*| = 42$. Tedy $P_{\text{char}} = 22/25 = 0,88$, $R_{\text{char}} = 22/42 \approx 0,52$, $F_{1,\text{char}} \approx 0,65$.

Na rozdíl od přístupu v bodě 1 dostane Systém 2 nenulové skóre, které odpovídá tomu, že část citátu byla detekována správně.